File System and Utilities

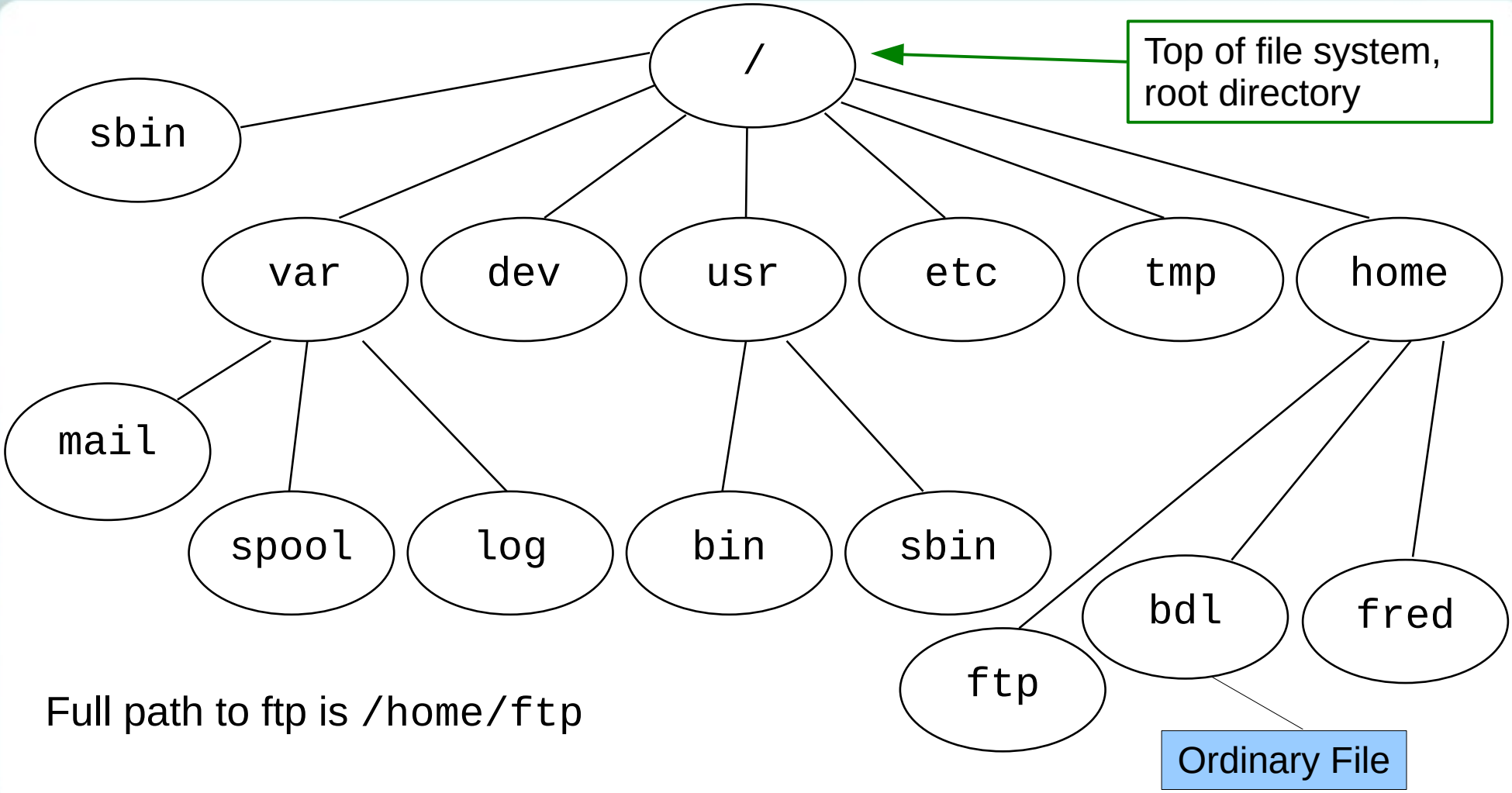
Today we want to learn:

- Linux File System, Chapter 6
- Utilities, Chapter 7
- X, part of Chapter 8 – for you to read

Chapter 6: The Linux Filesystem

- Hierarchical file system
  - top of hierarchy is root directory named `/`
- The file system contains:
  - ordinary (or plain) files
  - directories
  - special files
- Provides support for linking
- Provides support for file level protection

File System



Full path to ftp is `/home/ftp`

Unix File System (cont'd)

- Whole file system can consist of several "filesystems" each residing on a different disk partition
  - Possibly with different formats such as NTFS, ext3, FAT
- Each "filesystem" is a tree
- Can mount one "filesystem" onto another
  - for example, a cdrom could be mounted under /mnt/cdrom/

What Precisely is a File?

- inode + data blocks
- inode is a structure containing
  - file owner
  - file group
  - access modes
  - timestamps: last accessed, modified, inode update
  - size of file (in bytes)
  - number of links to a file
  - type of file (plain, directory, etc)
  - array of pointers to data blocks

What is a File? (cont'd)

- Note: filename is not contained in inode
  - Not part of the file at all!
- Each inode (in a given disk partition) is uniquely identified by an inode number
  - i-number

Referencing and Naming Files

- Case sensitive
- Almost any character is OK in a filename
  - Non-UNIX filesystems have more restrictions
- Up to 255 characters in length
- Filename extensions are part of the filename
  - Multiple filename extensions are OK
- *Absolute* pathname (starts with `/`)
  - Otherwise is *relative* to working directory
- Files start with `.` (period) are “hidden”

Naming Files: Good Practice

- Restrict characters to `a-zA-Z0-9_.,`
  - Name should be descriptive but not too long
- Do not use spaces, use `.` or _ instead
  - Spaces are special characters so will need to be quoted
    - E.g. my\ file or ‘my file’
  - This is annoying to the user and may break some code
  - Probably this is why Microsoft likes to do it
- See text for examples of common extensions

What is a Directory?

- File containing directory entries
- Directory entry: inode number + filename
- Directories always contain 2 files, `..` and `.`
- The directory you are “in” is called the “working directory” (or “current directory”)
- Home directories can be referenced with ~
  - In some shells only
- Created with `mkdir`, removed with `rmdir`

Special File Types

- 7 different types of Unix files
  - - plain files
  - d directories
  - b block special file (block I/O)
  - c character special file (char I/O)
  - l symbolic link
  - s socket
  - p named pipe

The `ls` Command

- All i-node information can be obtained with the `ls` command.
- Executing `ls -la` provides:

```
$ ls -la
total 212
drwxr-xr-x   9 neil  case    1344 Feb 11 15:09 .
drwxr-xr-x  70 root  case    1120 Feb 05 02:27 ..
-rw-------   1 neil  case      80 Feb 01 13:27 .cshrc
-r--r-----   2 neil  case     512 Feb 11 14:43 piano
----r-xrwx   1 neil  case     244 Feb 03  3:20 sax
lr--r-----   1 neil  case     512 Feb 11 14:47 a -> sax
drw-rw--rw   2 neil  case     512 Feb 11 14:43 piano
```

Access Permissions

- UNIX identifies the owner of a file and has permissions which control access
- File permissions control what can be done to the file
  - by the owner (user)
  - by others in the same group
  - by everyone
- Access Control Lists (ACLs) provide finer control

Users and Groups

- UNIX has "users”
  - usually each login account creates a user
  - each user has a numeric value and is given a name
- UNIX has "groups”
  - groups have numeric values and are given names
- users belong to one or more groups

File Ownership

- Each file is owned by a user (user ID of the file)
- Each file is associated with a group (group ID of the file)
- The group ID of the file is usually a group the owner belongs to
- Most linux distributions default to each user having their own group
  - can be changed by the administrator

Access Permissions

- 3 operations on a file: read, write & execute
  - for a directory, execute means searching the directory to resolve a pathname
- Access permission depends on two things
  - type of access requested
  - who is requesting the access
- Three levels of access permissions
  - user (owner), group, other

Access Permissions (cont'd)

- The access mode for a file is 9 bits long, and is displayed as a string of 10 characters
  - -rw-r--r--
- The superuser is not restricted by access permissions

Sample Access Modes

drwxrwxrwx	dr--r--r--
-rwxrwxrwx	-rwxr-xr-x
-rwx------	-rw-r-----
-rw-r--r--	d--------x

The chmod Command

- Use chmod to change the access permissions on a file
- Syntax: `chmod <mode> filename`
- Examples:

```
chmod 777 buddy
chmod 644 kula
chmod 503 chmod
chmod u+rw multisync
chmod a+x digger
```

Setuid and Setgid Permissions

- If execute file with setuid permission it takes the privileges of the file's owner
  - Usually executable runs under user's privileges
- Very dangerous if owner is root
  - Minimize these as security risk
  - Some programs need this to do there job
  - Never give shell scripts setuid
- Indication of setuid is s for owner in listing and chmod (setgid is s for group) in place of x

Access Control Lists (ACLs)

- Provides fine-grained control over which users can access specific directories and files
- Can reduce performance
  - Do not enable on file systems with system files
  - Enabled by default on ext2, ext3 systems
    - Disable by adding `no_acl` to options in /etc/fstab
- ACL comprises set of rules
  - Access rules (file or directory)
  - default rules (directory) flows to directory contents

Working with Access Rules

- `setfacl` modifies a file's ACL
  - `setfacl -m` *ugo:name:permissions filelist*
  - u or g indicates if name is user / group, o is other
  - -x (instead of -m) removes rules for user or group
  - m for ugo indicates mask limiting max permission
  - Separate multiple permission sections with ,
  - -d option makes default rule

Access Rule Examples

- setfacl -m u:sam:rw- report
- setfacl --modify u:sam:4,u:zac:rw- report
- setfacl -x u:sam report
- setfacl -d -m g:pubs:r-x, g:admin:rwx dir
- setfacl -m m::r-- report
- setfacl -m user:sam:6 report

The umask

- umask is a 9 bit mask that is used to set the initial file protection for newly created files
- It is a mask so it is the “inverse” of the desired maximum protection
- Display current setting by typing `umask`
- Set the umask with `umask <mode>`
  - `umask 022` sets protection to at most 755
  - `umask 027` sets protection to at most 750

Links

- A link is a mechanism which allows multiple filenames to refer to a single file on disk
- 2 types of links:
  - hard links
    - Original type of link
    - More limiting and less used than soft links
  - symbolic (or soft) links

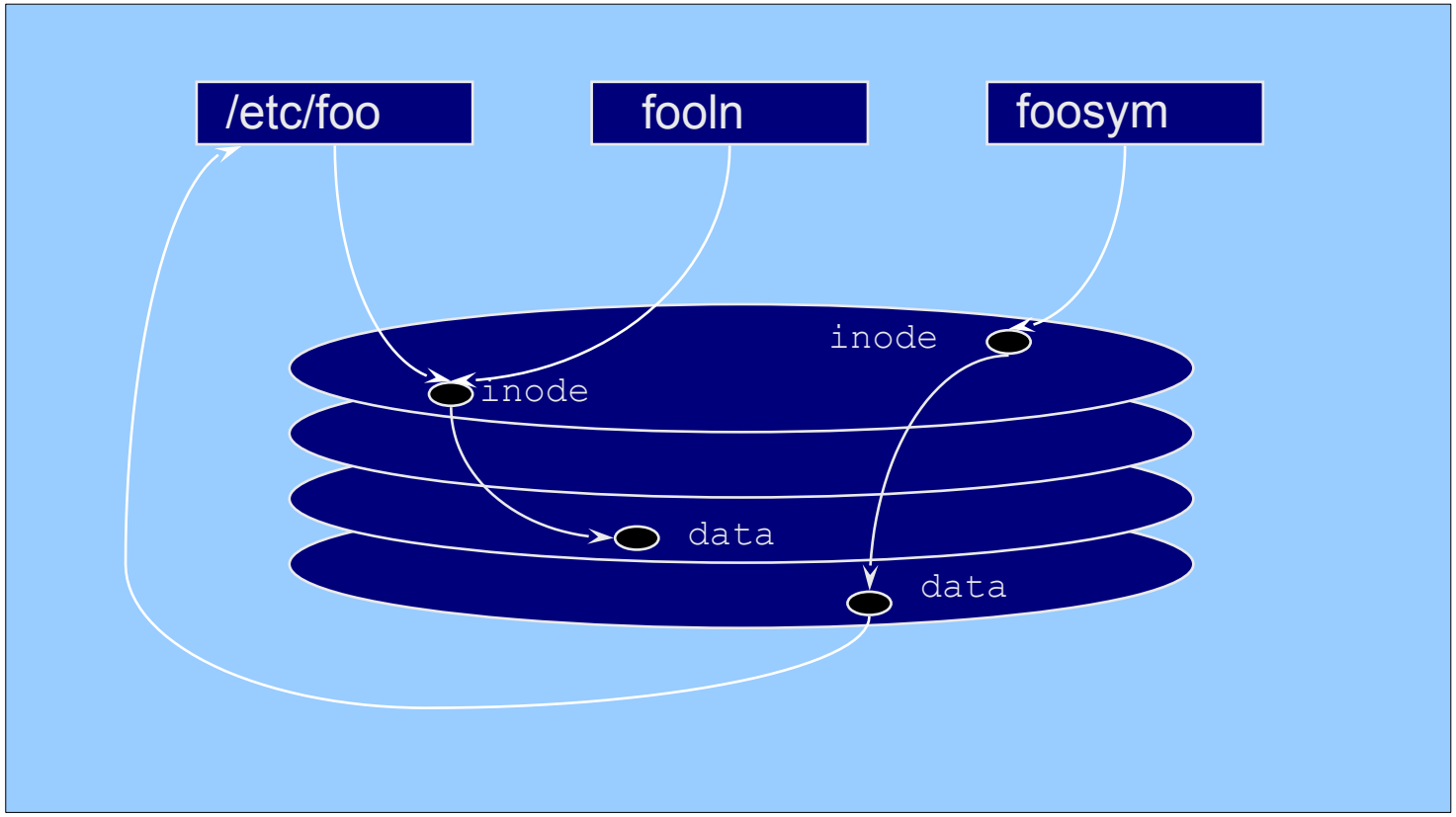
Hard Links

- Created with the `ln` command
  - eg: `ln /etc/foo fooln`
    - creates entry fooln, with same inode as /etc/foo
- A count is maintained in the inode indicating how many links there are to the file
  - Count starts at 1
- File not removed until all hard links deleted
- Hard link is considered to be a plain file
- Hard links cannot cross disk partitions

Symbolic Links

- Symbolic links are files pointing to other files
  - Have their own inode
  - Created with the `ln -s` command
- E.g.: `ln -s /etc/foo foosym`
  - Creates a new file, foosym, pointing to foo
  - foo's inode unaffected when foosym created
  - if delete /etc/foo, references to foosym generate an error (foosym points to nothing)
- Shown with type `l` files in directory listings
- Can cross disk partitions

Links (cont)



Commands To Know

- `cd`
- `chmod`
- `ln`
- `mkdir`
- `pwd`
- `rmdir`
- `rm`

Chapter 7: The Linux Utilities

- UNIX originally had no graphical user interface
  - Had not been invented
- All of tools ran from command line
  - Tools became very powerful and efficient
- Command line utilities often faster, more powerful, more complete then GUI alternatives
- If connecting remotely to a system you may only have command line utilities available
  - Need to know them

Special Characters

- The characters `&;|*?'"` `[]()$<>{}#/\!~` have special meaning to the shell
  - Avoid using before you know what they mean
- Space, tab and newline (called whitespace) separate tokens in a command
- Return character ends a command line and starts execution of the command

Quoting (Escaping) Characters

- Can get rid of character's special meaning by *quoting* (or *escaping*) the character
  - \ (backslash) quotes character following it
    - Except / is a directory separator in a path even if quoted
  - Pair of single quotes (‘) quote what is between
  - Control characters can be quoted by preceding with Ctrl-V
    - Such as erase (Ctrl-H) or line kill (Ctrl-U)

Basic Utilities

- `ls`
  - Lists the files in a directory
- `cat` *`filename`*
  - Displays the contents of *filename*
- `rm` *`filename`*
  - Deletes the file named *filename* (no warning)
- `hostname`
  - Displays name of system you are logged into

Pagers: `less` is `more`

- more *filename* or less *filename*
  - Displays *filename* a screen at a time
  - space displays next page
  - b displays previous page
  - q goes back to command prompt
  - h displays help
    - Can do fancy things like search for regular expression

More Utilities

- For all commands, TAB will try to complete the name of a command or file name
  - Multiple possibilities, second TAB shows list
- File operations
- echo/date
- Compression and archiving
- User and system information
- Communicating with other users

File Operations

- `cp` *source-file destination-file*
  - Copies the *source-file* to the *destination-file*
  - Overwrites destination if it exists (no warning)
- `mv` *source-file destination-file*
  - Renames *source-file* to *destination-file*
  - No warning if *destination-file* already exists
- File names can have periods and are case sensitive
  - More rules when discuss file system

Printing Files

- `lpr` *filename*
  - Places *filename* on the standard print queue
  - Can also have a list of file names, separated with spaces
- `lpq` lists the files waiting on the print queue
- `lprm` *number*
  - Removes the print job *number* from the queue
- `lpr -P`*printqueue filename*
- `a2ps` formats and prints file

File Utilities

- `grep` *expression filename*
  - Displays the lines in *filename* that contain *expression*
- `head` *filename*
  - Displays first 10 lines of *filename*
- `tail` *filename*
  - Displays last 10 lines of *filename*
- `sort` *filename*
  - Displays the contents of *filename* sorted by line

File Utilities

- `uniq` *filename*
  - Displays a file, omitting consecutive duplicate lines
- `diff` *filename1 filename2*
  - Displays a list of differences between filename1 & 2
  - Useful in preparing patches
- `file` *filename*
  - Displays the type of *filename*

| (Pipe): Communicates Between Processes

- Each UNIX command is run by a process
- A pipe connects the output of one process to the input of another process
  - More precisely, standard out to standard in
- Indicated by writing commands separated by |
  - `cat months | head`
  - `tail messages | sort | uniq | lpr`
- Used to combine several commands to accomplish a needed task

Miscellaneous Commands

- `echo` anything following
  - Displays the part of the command line following the keyword “`echo`”
  - Used in shell programs
- `date`
  - Displays time and date
- `unix2dos` *filename*, `dos2unix` *filename*
  - Converts text file between Mac/UNIX and DOS/Windows (different line terminators)

Compression Utilities

- `gzip` *filename*
  - compresses a file and adds a `.gz` suffix to the name
  - standard UNIX equivalent to zip
- `gunzip` *filename*
  - decompresses a file compressed with `gzip`, removes the `.gz` suffix
- `zcat` *filename*
  - decompresses a file and displays the resulting file to the terminal

Newer Compression Utilities

- `bzip2` filename
  - Compresses file and adds `.bz2` suffix to the name
  - More efficient than `gzip` but not as commonly used
- `bunzip2` and `bzcat` analogous to `gunzip` and `gzcat`
- Occasionally see `compress` used which results in a file ending with `.Z` (use `uncompress`)

Archiving

- `tar`
  - utility which handles archives of directories and files
    - see man pages for `tar` (`jar` is based on this)
  - `tar -cf` *filename list-of-files-to-archive*
    - Creates archive
  - `tar -tf` *filename*
    - Lists the contents of the archive
  - `tar -xf` *filename*
    - Recreates the directories and files from the archive

Common `tar` options

- `-j`
  - Compress when archiving, uncompress when dearchiving (using bzip2: extension .tar.b2 or .tbz)
- `-z`
  - Compress when archiving, uncompress when dearchiving (using gzip: extension .tar.gz or .tz)
- `-p`
  - Preserve permissions
- `-v`
  - verbose

Finding Commands

- `whereis` *command* and `apropos` *keyword*
  - Look for a command and lists locations
  - `whereis` looks in standard places where commands may be found (by command name)
  - `apropos` searches a database of commands (by keyword)
- *`which` command*
  - Shows location of command which will be run by looking through PATH

Finding Files

- `locate` *filename*
  - Uses updatedb to locate file on system containing *filename* in its name
  - Good for quick search where only need to find the file
  - Usually updatedb will be updated once a day
- A much more powerful utility, `find`, allows searching for files by multiple attributes
  - And executing conditional code on each

Obtaining User Information

- `who`
  - Displays a list of users logged in to a machine
- `finger` *username*
  - Displays information about user *username*
  - Information which is displayed is in a file called `.plan` located in the user's home directory
  - Some systems disable this as a security leak
- `w`
  - Displays list of users logged in and resources used

Communicating with Others

- `write` *`username`*
  - Sends messages to another user on the same machine
  - Second user can establish 2-way communication by issuing a write command to the first user
  - (Can also use GUI messaging tools)
- `mesg n`
  - Disables receiving these messages
  - `mesg y` to enable again

Electronic Mail

- Many tools to send/receive email on linux.
  - mail, elm, mh, xmh, exmh, Mozilla/Thunderbird, emacs
- Many people use pine
- GNOME and KDE have their own graphical mail clients
- You can always use gmail via a browser

Use `emacs` or `vim` to Edit Files

- Linux has several powerful file editors
  - Most capable being `emacs` and `vi`
- Text has tutorial on using `vi`
- Arguably more powerful is `emacs`
    - Comes with built-in tutorial
- Also are numerous notepad-like editors
- Can also use smaller editors: nano or pico

Power Commands

- These are useful in a variety of contexts
- Some will be covered in the lab assignments
  - find
  - awk
  - sed
  - perl

Chapter 8??: Linux GUIs: X, Gnome and KDE

- This section covers a bit of the technical underpinning behind the GUI interface
- Specific details about GNOME and KDE are left for the text and online documentation
- The main idea is the client-server aspect that allows running programs on one system and displaying output on another

Components of a GUI

- Look and feel of windows similar to other GUIs
  - root window, graphical things on top
- Window Manager
  - twm, vtwm, fvwm, motif, enlightenment
  - controls look and feel of basic GUI
- Desktop Manager
  - K Desktop Environment, GNOME
  - may run on top of window manager
  - allows drag and drop applications

The X Stack

<table>
<tr><td colspan="5">Graphical applications</td></tr>
<tr><td>GTK</td><td>Qt</td><td>Tk</td><td>Motif</td><td>Other toolkits</td></tr>
<tr><td colspan="5">Xlib</td></tr>
<tr><td colspan="5">X server</td></tr>
<tr><td colspan="5">Linux kernel</td></tr>
</table>

X Window System

- Portable and flexible
  - Available on Macs and PCs running MS Windows, NT, or OS/2 as well as UNIX
- Designed for a network workstation
- Can run X application on one computer and display results on another
- Includes X Toolkit for graphics
  - Widget sets to draw controls

The X Window System

- Invented in 1984 at MIT
- Network windowing system
  - Can run an X app on one machine & have it displayed on another machine running X
- Based on the client-server model
  - X server runs on the local machine
    - provides keyboard, mouse and windows to applications
  - X applications are the clients
    - may run on remote machines
    - display on machine running the X server

Starting the X Server

- Linux can start in text mode (run level 3) or graphics mode (run level 5)
- In graphics mode, then a program called xdm presents login window
  - Or equivalent, such as kdm
  - Then X server is already started
- If in text mode, command `startx` starts the X Server
  - sets variable `DISPLAY` to identify the server

Remote Computing with Local Displays

- Decouples application from display
- X server runs on local machine
- Application uses `DISPLAY` environmental variable to identify where to display its output
  - don’t have to worry about this for apps on local machine
  - usually the `DISPLAY` variable is set up in a login script when logging on to another machine

X Window System

- X server is responsible for all input from the console (keyboard, mouse, etc) and for drawing to the screen
- X clients ask the X server to draw to the screen
- X clients and X server communicate using the X protocol
  - run /usr/bin/xev to see what it looks like
- X does *not* specify look & feel
  - window manager does this

Security and X

- To connect to remote X server need to modify two security features
  - `X -nolisten tcp` option prevents tcp connections to X server (allow this)
    - GNOME: Uncheck *Deny TCP connections to Xserver* in System>Administrator>Login Window
    - KDE3.5: Uncheck *Deny TCP connections to Xserver* in System>Login Window (security tab)
    - KDE4: unknown
  - xhost +hostname allows hostname to connect to Xserver

Example

- Using the "display" option to have xterm display to another machine running X

```
$ xterm -display L327S26:0.0
```

  - L327S26 is the machine’s host name
  - In 0.0, the first zero is the server number
  - the second zero is the screen number
  - an X server can manage more than 1 screen!

Example

- Set the DISPLAY environment variable to tell the X apps where to display themselves

```
$ export DISPLAY L327S26:0.0
$ xterm&
```

- This affects the programs started from the command window setting the DISPLAY variable
- xterm will try to connect to server 0 on L327S26

DISPLAY

- If DISPLAY is set to :0.0
  - means X clients should connect to the server using UNIX domain sockets
  - in this case, clients must run on the same machine as the server
- If DISPLAY is set to something like L327S26:0.0
  - clients will connect to L327S26 using TCP/IP

Access Control

- Previous examples will work only if there is no access control
- Any client that successfully connects to your X server has complete control over the display
- Access control is needed to deny unwanted connection requests to your X server
- 2 forms of access control
  - host-based access control with xhost
  - access control with MIT-MAGIC-COOKIE-1

xhost

- Gives another *machine* complete access to your display

```
xhost +hostname     xhost -hostname
xhost +             xhost -
xhost
```

- Allows access by another users logged into the same remote machine
- Obvious security concerns (what are these?)

MIT-MAGIC-COOKIE-1

- X server must be configured to use this
- Machine-readable code (called a magic cookie) put in file called `.Xauthority` in your home directory when you start X
- X client must present this code before it is allowed to connect to your X server
  - no problem if the X client can access `.Xauthority` in your home directory
  - `xauth` sends the magic cookie to a remote host for clients run on another machine

xauth

- Assume X server is running on the local machine (sgi7) & you are logged into the remote machine (sgi8) too
- To send the magic cookie to the remote machine (sgi8), type in the local machine (sgi7):

```
xauth extract - $DISPLAY | \
rsh remotehost xauth merge -
```

- Alternatively, type in the remote machine (sgi8):

```
rsh sgi7 xauth extract - sgi7:0 | xauth merge -
```

- See the man pages for xauth

Shutting Things Down

- Graphical applications should have ways of gracefully shutting themselves down
- If these do not work
  - CONTROL-ALT-ESCAPE should give you a skull and crossbones cursor to kill the application
    - click on its window
    - can also kill the process running it
  - If X needs to be restarted (all windows frozen)
    - log into the system from another terminal and kill the process running Xorg
    - try CONTROL-ALT-BACKSPACE to kill and restart X

X Window Managers

- Several available on Linux
- Depends on user tastes
- fvwm (Virtual Window Manager)
- Motif (mwm)
- Metacity (GNOME default)
- KWin (KDE default)
- Compiz (compositing window manager)
- Enlightenment

Selecting Active Window

- Two main ways of selecting active window
  - called giving the window focus
  - this window receives all characters typed
- click-to-focus
  - window last clicked on retains focus no matter where mouse moves
- focus-follows-mouse
  - window with pointer currently over it gets all characters typed

Copying and Pasting Text

- Select text to copy
  - position pointer to left of text
  - hold down left mouse button and drag over text
  - release left mouse button
  - text is highlighted
- Paste text in new location
  - place pointer where the text is to go (possibly in other window)
  - click middle mouse button to paste

Linux Desktop Environments

- Desktops run on top of a windows manager
- Currently a desktop war between Gnome and KDE
  - Been going on for years, now
- Both available on the lab clients
- GNOME better supported by RedHat
- KDE seems to be gaining ground
- Your choice which to use

KDE

- Project started in 1996
- Defined high quality, high performance desktop and user graphical shell
- Large collection of consistent applications
- Set of tools for desktop application
- Written in C++ using Qt framework
- Very modular design

GNOME

- Default for Redhat/Fedora
- Uses GTK for drawing
  - written in C
- Tends to promote a simpler, cleaner user interface
- Most of the GUI tools you are used to are available under GNOME
  - KDE too
  - these are convenient after you have mastered the text-based commands

Window Manager

- Full-featured configuration interface
- Smart window placement
  - places windows to avoid clutter
- Magnetic window borders and desktop
  - turn on “active desktop borders”
- Themes
- Virtual desktops

Panel/Launcher

- Fast access to most applications (K menu)
- Docking area, fast lock, desktop switching
- Advanced task-list (right click for options)
- Auto-hide functionality
- Disk naviigator, with history, quick browsing

Control and Help

- Control center
  - consistent configuration tools
  - on-the-fly configuration
- Help center
  - HTML-based
  - unifies KDE documentation, man/info pages
  - hooks to applications

Summary

- Graphical User Interfaces in Linux are modular, layered systems
  - much more than in MS Windows
- Users get familiar with the specific tools available on their preferred desktop system
  - `kfind` finds files under KDE
  - `beagle` finds files under GNOME
  - `find` finds files from the command line
- Most of the GNOME tools can work under KDE
  - and vice versa
- Be sure you can use the command line first
  - and know how to open a terminal window from your desktop!